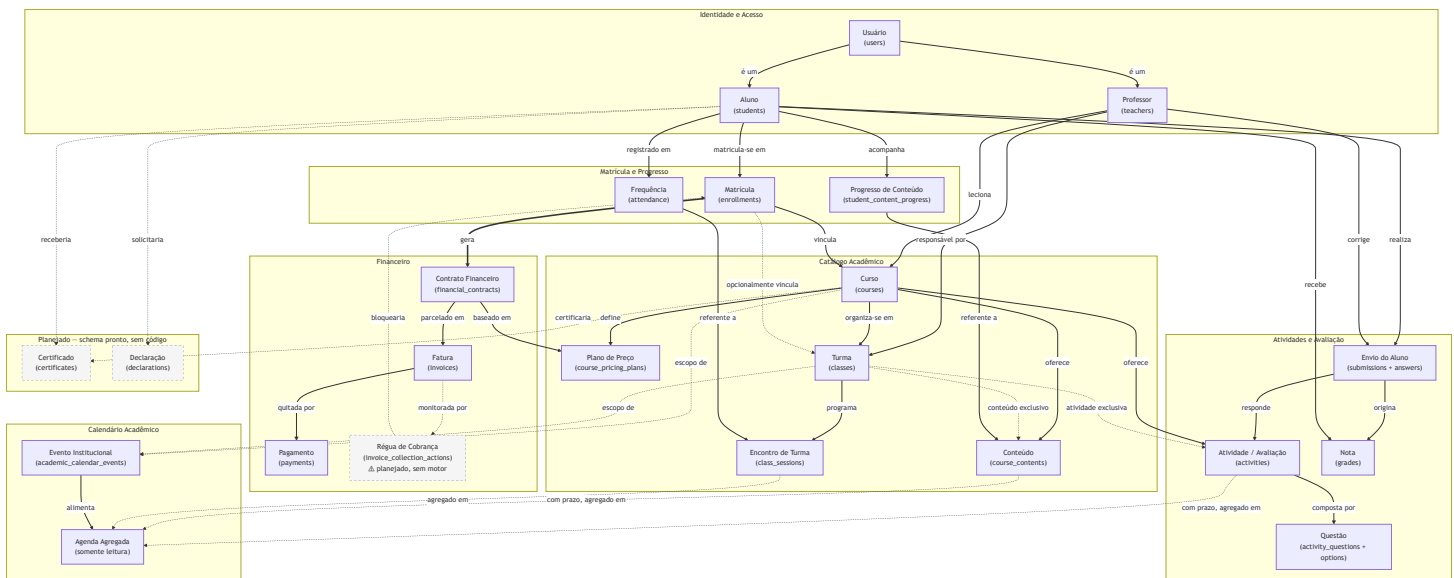


Diagrama — Modelo de Domínio

Diferente do ERD (`database-erd.md`), que mostra tabelas e colunas, este diagrama mostra os **conceitos de negócio** do CourseHub e como se agrupam em áreas de domínio (bounded contexts informais). Os nomes usados aqui são os mesmos das tabelas reais — nenhum conceito foi inventado — mas a organização é por significado de negócio, não por schema.



Legenda: setas sólidas (--> , ==>) representam relações ativas e implementadas. Setas tracejadas (-. ->) representam relações condicionais/opcionais (dependem de um campo nullable, como `class_id`) ou — no caso do bloco "Planejado" — relações que o schema modela mas que não têm código de aplicação por trás ainda.

Que problema este diagrama esclarece

O ERD responde "quais colunas apontam para quais tabelas". Este diagrama responde uma pergunta diferente e mais útil para quem está aprendendo o produto: "quais são os conceitos de negócio do CourseHub, e como um aluno, uma matrícula, uma atividade e um contrato se relacionam na cabeça de quem usa o sistema, não na cabeça de quem escreveu o SQL". É o documento certo para explicar o domínio a alguém que vai trabalhar em regras de negócio, não em queries.

Como interpretar os elementos

- Cada subgrafo é uma área de domínio coesa — os nomes (IDENTIDADE , CATALOGO , MATRICULA , AVALIACAO , FINANCEIRO , CALENDARIO) não são bounded contexts formais no sentido de DDD com

fronteiras de consistência transacional definidas em código, mas descrevem honestamente como o código já está organizado (cada um mapeia a uma pasta em `backend/services/`).

- O bloco `FUTURO` , com preenchimento cinza tracejado, existe para que ninguém confunda "está no diagrama" com "está implementado". `ReguaCobranca` também recebe essa marcação mesmo estando no subgrafo `FINANCEIRO` , porque seu status real (schema sem motor de execução) é o mesmo dos outros dois.
- `Envio` representa juntas as tabelas `submissions` e `submission_answers` — no domínio de negócio, "o envio de um aluno" é um conceito só, ainda que sejam duas tabelas por razão de normalização (cabeçalho + respostas por questão).

Que decisões arquiteturais este diagrama evidencia

- **Matricula é o nó de articulação entre `CATALOGO / MATRICULA` e `FINANCEIRO`** : nenhuma cobrança existe sem uma matrícula (`financial_contracts.enrollment_id` é obrigatório e único). Isso reflete a regra de negócio real: você não cobra alguém que não está matriculado.
- **AgendaAgregada não é uma entidade de dados, é uma capacidade de leitura**: propositalmente desenhada sem tabela própria associada, para deixar visualmente óbvio que o calendário não guarda nada, só combina.
- **Turma aparece com relação tracejada para `Conteudo` e `Atividade`** : reforça visualmente que "pertencer a uma turma" é a exceção (`class_id` preenchido), não a regra — o padrão geral já é o padrão default (`class_id NULL`).

Trade-offs que este modelo representa

- Modelar `Envio` e `Nota` como conceitos separados (em vez de "resultado da atividade" único) exige duas escritas coordenadas ao corrigir uma atividade, mas permite que um envio exista em estados intermediários (`draft` , `submitted`) antes de qualquer nota existir — importante para o fluxo real de "aluno começa a responder, salva, volta depois".
- Ter `Certificado / Declaracao / ReguaCobranca` desenhados no domínio antes de implementados é uma aposta de planejamento: o modelo de dados já reserva o espaço conceitual certo, evitando um redesenho de schema quando a funcionalidade for construída — mas corre o risco de o desenho ficar desatualizado frente a requisitos reais que só apareçam no momento da implementação.

Como este diagrama deve evoluir

Quando `certificates / declarations` /a régua de cobrança ganharem implementação real, mova-os do bloco `FUTURO` para o subgrafo de domínio apropriado e remova a marcação `planejado`. Se um novo conceito de negócio surgir que não mapeie limpo para uma tabela existente (por exemplo, "Turma Concluída" como um evento, não um estado), prefira adicionar o conceito a este diagrama primeiro e só depois decidir seu formato de persistência — este documento deve continuar sendo a leitura de domínio, não um espelho atrasado do ERD.